Plano de Testes & Roteiro de Debugging CLI

Guia de Validação Iterativa para Nível 2 (Misto) e Nível 3 (Técnica)

> [NOTA] **Objetivo deste material:**
> Ajudar você a testar cada etapa de correção diretamente na linha de comando (CLI), entender as mensagens de erro dos scanners, diagnosticar causas-raiz e raciocinar sobre a solução sem depender de tentativa e erro.
> **Aviso importante:** Este plano contém **métodos de teste, sintomas de erro e perguntas socráticas**, e não o código pronto da resposta. A engenharia de solução deve ser construída por você!

---

1. O Ciclo DevSecOps de Depuração Local

Não suba commits às cegas esperando o GitHub Actions rodar. Siga o ciclo ágil de feedback rápido no seu terminal:

```
1. DIAGNÓSTICO     2. HIPÓTESE TÉCNICA   3. CORREÇÃO
(Rodar Scanner)         (Ler o Erro / Log)         (Ajustar Código)



          4. REVALIDAÇÃO CLI
                          (Testar se Zerou)
```

Você pode validar suas alterações por duas vias locais:
- **Via Rápida (Recomendada - Sem Docker):** Usando o validador nativo `python scripts/selfcheck.py --nivel 2` ou `--nivel 3`.
- **Via Profunda (Com Docker):** Executando os mesmos contêineres que rodam no GitHub Actions (Trivy, Semgrep, Hadolint).

---

2. Etapa 1 Higiene de Repositório & Segredos (F1 - Nível 2 e 3)

Comandos de Teste e Inspeção

```
# 1. Verificar se o Git ainda está rastreando o arquivo .env:
git ls-files .env

# 2. Verificar o status da área de preparação:
git status

# 3. Rodar a autochecagem local:
python scripts/selfcheck.py --nivel 2
```

Sintomas & Erros Comuns

O que você vê no terminal	Causa Provável
O comando `git ls-files .env` imprime `.env`	O arquivo `.env` já foi versionado no passado e continua rastreado no índice do Git, mesmo que você tenha adicionado `.env` ao `.gitignore`.
A aplicação Flask não inicia localmente (`Error: SECRET_KEY not found`)	Você deletou fisicamente o arquivo `.env` da sua máquina em vez de apenas desindexá-lo do Git.
O script de autoavaliação aponta falta de `.env.example`	O repositório precisa fornecer um arquivo modelo com valores fictícios para outros desenvolvedores.

Dicas Iterativas de Raciocínio (Como resolver)

1. **Pense sobre o índice do Git vs. Sistema de Arquivos:**
   - O `.gitignore` instrui o Git a ignorar arquivos **não rastreados**. Se um arquivo já foi commitado uma vez, o `.gitignore` sozinho **não** o remove do histórico.
   - *Pergunta:* Qual flag do comando `git rm` remove o arquivo da área de rastreamento do Git mantendo o arquivo intacto no seu disco rígido? *(Dica: consulte `git rm --help` e procure pela opção de cache).*
2. **Modelo de Configuração (`.env.example`):**
   - Como um novo membro da equipe saberá quais variáveis de ambiente a aplicação exige sem olhar o código?
   - Crie um arquivo com as mesmas chaves (`SECRET_KEY` , etc.), porém com valores explicativos genéricos (ex: `"altere-em-producao"`).

---

3. Etapa 2 SCA & Compatibilidade de Bibliotecas (F2 - Nível 2 e 3)

Comandos de Teste e Inspeção

```
# 1. Testar manifesto com Trivy FS via Docker:
docker run --rm -v "${PWD}:/src" aquasec/trivy:0.51.2 fs --severity HIGH,CRITICAL --scanners vuln /src

# 2. Ou testar via autochecagem:
python scripts/selfcheck.py --nivel 2
```

Sintomas & Erros Comuns

O que você vê no terminal	Causa Provável
Trivy relata: `gunicorn Fixed Version: 23.0.0`	O Gunicorn instalado (`20.1.0`) tem vulnerabilidades de request smuggling e precisa ser promovido para a versão indicada.
Trivy relata falhas no `urllib3` e `werkzeug`	Você atualizou apenas o Gunicorn, mas manteve as outras bibliotecas antigas.
Ao rodar testes com `pytest` : `ImportError: cannot import name 'url_quote' from 'werkzeug.urls'`	O `werkzeug` foi atualizado para uma versão moderna (3.x), mas o `flask` continuou na versão antiga (`2.2.5`), que tentava importar funções descontinuadas.

Dicas Iterativas de Raciocínio (Como resolver)

1. **Interprete a coluna `Fixed Version` do relatório:**
   - Para cada pacote listado na tabela do Trivy, localize a versão mínima indicada na coluna de correção.

2. **Entenda o acoplamento do ecossistema Pallets:**
   - O Flask é construído sobre o Werkzeug. Se você atualiza a versão maior do Werkzeug para mitigar CVEs críticas, a versão do Flask também deve acompanhar (ex: Flask 3.x com Werkzeug 3.x).
   - *Ação:* Abra `requirements.txt` e atualize em conjunto os pacotes correlacionados, reexecutando o scanner até obter `Total: 0`.

---

4. Etapa 3 Linter & Hardening do Dockerfile (F4 - Nível 2 e 3)

Comandos de Teste e Inspeção

```
# 1. Validar sintaxe e boas práticas do Dockerfile com Hadolint:
# (Linux / Mac / Git Bash)
docker run --rm -i hadolint/hadolint:v2.12.0 < Dockerfile

# (PowerShell Windows)
Get-Content Dockerfile | docker run --rm -i hadolint/hadolint:v2.12.0

# 2. Compilar a imagem para teste local:
docker build -t aurora-app:test .

# 3. Analisar vulnerabilidades da imagem compilada com Trivy Image:
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:0.51.2 image --severity HIGH,CR
```

Sintomas & Erros Comuns

O que você vê no terminal	Causa Provável
Hadolint: `DL3007 warning: Using latest is prone to errors...`	A linha `FROM python:latest` utiliza uma tag mutável. Deve-se fixar uma versão explícita e minimalista (ex: `python:3.11-slim`).
Trivy Image relata dezenas de vulnerabilidades em pacotes Debian	A imagem base padrão traz compiladores e utilitários obsoletos do SO.
Ao iniciar o contêiner: `sqlite3.OperationalError: attempt to write a readonly database`	O contêiner agora roda como usuário sem privilégios (`USER nonroot`), mas o diretório onde o SQLite grava (`/app/instance`) pertence ao usuário `root`.
Hadolint avisa ausência de checagem de saúde	O Dockerfile não possui a instrução `HEALTHCHECK`.

Dicas Iterativas de Raciocínio (Como resolver)

1. **Os 4 Mandamentos do Hardening:**
   - Imagem base mínima (evite Debian full, use slim ou alpine).
   - Multi-stage build (separe o estágio que roda `pip install` do estágio final de execução).
   - Usuário não-root (`useradd -u 10001 -m nonroot` e instrução `USER nonroot`).
   - Monitoramento (`HEALTHCHECK --interval=30s ...`).
2. **Resolvendo o erro de permissão do SQLite:**
   - *Pergunta:* Antes de trocar o usuário com `USER nonroot`, quais comandos `RUN` você precisa executar para garantir que a pasta da aplicação e a pasta de instância pertençam ao novo usuário? *(Dica: investigue o comando `chown -R nonroot:nonroot`).*

---

5. Etapa 4 SAST & Injeção de SQL (F3 - Nível 3)

Comandos de Teste e Inspeção

```
# 1. Rodar Semgrep SAST localmente:
docker run --rm -v "${PWD}:/src" semgrep/semgrep:1.78.0 semgrep --config auto /src/app

# 2. Executar a suíte de testes unitários:
pytest -v tests/test_smoke.py

# 3. Ou rodar a autochecagem local:
python scripts/selfcheck.py --nivel 3
```

Sintomas & Erros Comuns

O que você vê no terminal	Causa Provável
Semgrep: `python.lang.security.audit.formatted-sql-query`	A função `search_clients` continua usando formatação de string (f-string ou `%`) para montar a query SQL.
Teste unitário `test_search_clients` falha ou não encontra ninguém	O caractere `%` do operador `LIKE` foi colocado entre aspas dentro da string SQL junto com o placeholder (`LIKE '%?%'`), o que faz o SQLite procurar pela interrogação literal.

Dicas Iterativas de Raciocínio (Como resolver)

1. **Como a DB-API do Python trata placeholders:**
   - No SQLite (`sqlite3` / DB-API), o caractere de parâmetro seguro é o `?` puro.
   - A query deve ser estática: `"SELECT * FROM clients WHERE full_name LIKE ? ORDER BY id"`.
2. **Onde colocar os wildcards `%`:**
   - Os caracteres `%` pertencem ao **dado**, não à instrução SQL.
   - *Pergunta:* Como você passa o argumento na tupla de parâmetros do método `.execute()` de modo que o valor da variável receba os `%` antes de ser vinculado ao banco? *(Dica: passe `(f"%{name}%",)` como segundo argumento).*

---

6. Etapa 5 IaC, Menor Privilégio & Headers DAST (F5, F6, F7 - Nível 3)

Comandos de Teste e Inspeção

```
# 1. Validar a sintaxe do Docker Compose:
docker compose config

# 2. Testar resposta da aplicação e inspecionar cabeçalhos HTTP com curl:
curl -I http://localhost:5000/health

# 3. Rodar testes automatizados:
pytest -v tests/test_smoke.py
```

Sintomas & Erros Comuns

O que você vê no terminal	Causa Provável
ZAP acusa `Missing Anti-clickjacking Header` ou `X-Content-Type-Options Header Missing`	Os cabeçalhos de segurança não foram adicionados a todas as respostas HTTP da aplicação.
Ao cadastrar um novo usuário na rota `/register`, ele já aparece como admin	A função `create_user` em `app/db.py` mantém o parâmetro padrão `role="admin"`.
Qualquer usuário autenticado consegue alterar papéis em `/admin/users/<id>/role`	A rota tem apenas `@auth.login_required`, sem verificar se o usuário logado possui perfil de administrador.
Docker Compose monta `/var/run/docker.sock` ou define `privileged: true`	Violação de IaC: contêiner com capacidade de controle sobre o daemon Docker do hospedeiro.

Dicas Iterativas de Raciocínio (Como resolver)

1. **Headers de Segurança Globais no Flask (F5):**
   - Em vez de adicionar headers manualmente em cada função de rota, use o mecanismo de interceptação do Flask que executa após toda requisição:
   `@app.after_request`
   - Dentro dessa função, adicione ao objeto `response.headers` os cabeçalhos recomendados pela OWASP (`X-Frame-Options`, `X-Content-Type-Options`, `Content-Security-Policy`, etc.).
2. **Governança e RBAC (F6):**
   - Mude o papel padrão de criação para `"client"`.
   - Crie uma verificação (ou decorador customizado `@admin_required`) que consulta o papel do usuário na sessão antes de permitir operações administrativas.
3. **Hardening de Orquestração (F7):**
   - Remova privilégios desnecessários do `docker-compose.yml`: elimine `privileged: true`, exclua montagens do socket do Docker e use volumes nomeados específicos para o SQLite.

---

7. Roteiro de Verificação Final Pré-Push

Antes de enviar seus commits para o GitHub:

1. [ ] **Verifique seu status local:**
   `bash git status`
   Certifique-se de que nenhum arquivo indesejado (como `.env` ou `__pycache__`) está pendente.
2. [ ] **Execute os testes unitários:**
   `bash pytest -v`
   Todas as asserções devem passar (`6 passed`).
3. [ ] **Rode a autochecagem do nível escolhido:**
   `bash python scripts/selfcheck.py --nivel 2 # Para Nível 2 python scripts/selfcheck.py --nivel 3 # Para Nível 3`
   O resultado deve ser `[OK]` em todas as falhas aplicáveis com código de saída 0.
4. [ ] **Abra a aba Actions no GitHub após o push:**
   - Acompanhe a execução dos jobs. Se algum step falhar, utilize as técnicas deste guia para ler o log do scanner correspondente, formular a hipótese técnica e iterar localmente!